

Assignment 1 – Explore n8n Case Studies

Dineshkumar

1. Fullscript (Healthcare Platform)

Problem / Use Case

- Fullscript handles **healthcare supplement prescriptions and patient workflows**
- Needed to:
 - Automate internal operations
 - Integrate multiple systems (CRM, support, internal tools)
- Challenge:
 - Too many manual processes
 - Engineering team overloaded with integration requests

Where & Why n8n

- Used as an **internal automation layer**
- Key reasons:
 - Reduce dependency on core engineering team
 - Enable **ops teams to build workflows**
 - Self-hosting → important for **healthcare data privacy (HIPAA-like concerns)**

Workflow Implementation

Typical workflows included:

- Trigger: Event (new order / support request / internal action)
- Steps:
 - Fetch data from internal systems
 - Transform/validate data
 - Trigger downstream systems (CRM, notifications, support tools)
 - Log activity for audit

Example:

- New patient order →
 - Validate prescription
 - Notify pharmacist/team
 - Update CRM
 - Send confirmation email

Real-World Impact

- Reduced engineering workload significantly
- Faster internal operations
- More control to non-dev teams

Tester Insight

- Critical areas:
 - **Data privacy validation**
 - Workflow audit logs
 - Error handling in patient-related workflows
 - Access control (who can modify workflows)

2. Flow AI (Conversational AI Platform)

Problem / Use Case

- Flow AI builds **chatbots and conversational workflows**
- Needed:
 - Integration with multiple APIs/tools
 - Flexible backend orchestration for conversations

Challenge:

- Hard to maintain custom backend logic for each integration

Where & Why n8n

- Used as a **backend orchestration engine for chatbot actions**
- Why:

- Easy webhook + API chaining
- Rapid workflow creation for new chatbot features
- Reduced custom backend coding

Workflow Implementation

- Trigger: User input via chatbot
- Steps:
 - Send input to NLP/AI model
 - Determine intent
 - Call relevant APIs (CRM, payment, DB)
 - Return response to chatbot

Example:

User: “Check my order status”

- Chatbot → n8n webhook
- n8n:
 - Extract intent
 - Call order API
 - Format response
- Return to chatbot UI

Real-World Impact

- Faster chatbot deployment
- Easier integration with external systems
- Scalable conversation workflows

Tester Insight

- Focus on:
 - **Intent mapping accuracy**
 - API chaining failures
 - Response latency (chat UX sensitive)
 - Handling ambiguous user inputs

3. Musixmatch (Music Platform)

Problem / Use Case

- Musixmatch manages **lyrics, metadata, and music content at scale**
- Needed automation for:
 - Content processing
 - Data synchronization across platforms
 - Partner integrations

Challenge:

- Large-scale data workflows → complex + repetitive

Where & Why n8n

- Used for **data pipeline automation**
- Why:
 - Handles large-scale integrations
 - Reduces custom scripting
 - Flexible scheduling + event-based triggers

Workflow Implementation

- Trigger: New content / scheduled job
- Steps:
 - Fetch music/lyrics data
 - Transform metadata
 - Sync with partner platforms (Spotify, Apple, etc.)
 - Validate data consistency
 - Log results

Example:

- New lyrics added →
 - Validate format
 - Map metadata

- Push to multiple platforms
- Notify stakeholders

Real-World Impact

- Faster content delivery
- Reduced manual intervention
- Improved data consistency across platforms

Tester Insight

- Critical validations:
 - **Data transformation accuracy**
 - Duplicate/override handling
 - Sync failures across platforms
 - High-volume performance

Assignment 2 – Explore & Build an n8n Template

Template: Answer multi-channel support queries with OpenAI RAG and Supabase

What problem this workflow solves

This template implements a **RAG (Retrieval-Augmented Generation)** support bot that:

- Ingests your support documentation (PDFs, docs, etc.) into a **vector database (Supabase)**.
- At query time, **retrieves the most relevant chunks** from your docs.
- Uses **OpenAI** to generate answers **grounded in your actual content**, not the model's general knowledge.
- Can be extended to **multi-channel** (web chat, Telegram, WhatsApp, email, etc.) by swapping the `trigger.triggerworkflow+2`

This directly addresses the common failure mode: “The bot confidently invents a refund policy that doesn’t exist” because it was never shown your real policy. RAG fixes that by giving the model your documents at the moment it needs them.[triggerworkflow](#)

High-level architecture: two workflows, not one

A complete RAG system in n8n is **two separate workflows**:`triggerworkflow+1`

1. **Ingestion workflow** (run when docs change)
 - Load documents → chunk → embed → store in Supabase.
2. **Chat/query workflow** (run per user question)
 - Receive question → embed query → retrieve relevant chunks from Supabase → pass to OpenAI → return answer.

The template you’re looking at is primarily the **chat/query workflow**, but it assumes you already have an ingestion pipeline that populated Supabase.`triggerworkflow+1`

Core nodes and why they’re used

Below is the typical node set you’ll see in this kind of template, with the purpose of each:

1. Chat Trigger (or Webhook / Messenger trigger)

- **What it does:** Starts the workflow when a user sends a message (via n8n’s built-in chat UI, a webhook, Telegram, WhatsApp, etc.).
- **Why:** This is your entry point. In a “multi-channel” setup, you’d have different triggers for different channels but the same downstream logic.`triggerworkflow+1`

2. AI Agent node

- **What it does:** Orchestrates the conversation. It can:
 - Call tools (like the vector store).
 - Use memory for follow-ups.
 - Decide when to search your knowledge base vs answer from general knowledge.
- **Why:** Using an **Agent** (instead of a simple chain) lets the system:
 - Choose whether to retrieve from your docs.
 - Potentially call other tools later (e.g., create a ticket, log to Sheets).

3. OpenAI Chat Model (sub-node of AI Agent)

- **What it does:** Provides the LLM (e.g., gpt-4o-mini) that generates the final answer.
- **Why:** This is the “brain” that turns retrieved context + question into a human-readable response.

4. Supabase Vector Store (as a tool for the AI Agent)

- **Operation mode:** “Retrieve Documents (As Tool for AI Agent)”.
- **What it does:**
 - Embeds the user’s question.
 - Searches Supabase for the most similar document chunks.
 - Returns those chunks as context to the agent.
- **Why Supabase:** It’s a persistent, production-ready vector store (PostgreSQL + pgvector). Unlike the in-memory store, it survives restarts and scales.
- **Key settings:**
 - **Tool name & description:** Critical. If the description is vague (“vector store”), the agent may ignore it and answer from its own knowledge. A good description: “Search our support knowledge base (policies, FAQs, product docs) before answering any product or policy question.”
 - **Top-k:** How many chunks to retrieve (commonly 4–6). More chunks = more context but higher token cost and potential noise.

5. Window Buffer Memory (optional but recommended)

- **What it does:** Stores recent conversation turns so follow-up questions (“What about refunds for that?”) make sense.

- **Why:** Without memory, each question is treated in isolation, making multi-turn conversations awkward.triggerworkflow+1

6. (Separate workflow) Ingestion pipeline nodes

Though not always visible in the chat template, you must have an ingestion workflow that uses:

- **Trigger:** Manual / Schedule / Google Drive watch.
- **Source node:** Google Drive, HTTP Request, File Upload, etc.
- **Extract from File:** Converts PDFs/DOCX to text.
- **Supabase Vector Store** in **Insert Documents** mode, with:
 - **Embeddings OpenAI** sub-node.
 - **Default Data Loader** sub-node.
 - **Text Splitter** sub-node (chunking).
- **Why:** This builds the index the chat workflow queries. Without it, the vector store is empty and the bot will answer from general knowledge (or invent things).triggerworkflow+2

How the flow solves the real-world problem

Step-by-step for a user question:

1. User sends: "What's your refund policy for digital products?"
2. **Chat Trigger** fires, passing the message to the **AI Agent**.
3. Agent reads its **tool description** and decides: "I should search the knowledge base."
4. Agent calls the **Supabase Vector Store tool**:
 - Query is embedded.
 - Supabase returns top-k relevant chunks (e.g., refund policy sections).
5. Agent receives chunks + original question.
6. **OpenAI Chat Model** generates an answer **grounded in those chunks**, e.g.:
"For digital products, refunds are available within 7 days if the download hasn't been used..."
7. Answer is sent back to the user via the original channel.
8. **Memory** stores this exchange so follow-ups ("What if I already downloaded it?") are coherent.triggerworkflow+2

This prevents hallucinated policies because the model is explicitly instructed (via system prompt + tool usage) to answer from retrieved context and say “I don’t know” if the knowledge base has no answer.triggerworkflow+1

Recreating the workflow yourself in n8n

Here’s how to build it from scratch, understanding each choice:

Step 1: Set up Supabase as your vector store

1. Create a Supabase project.
2. In the SQL editor, run the pgvector setup (create vector extension, documents table, and match_documents function) as per Supabase’s RAG quickstart.noderecipe+1
3. Note:
 - Table name (e.g., documents).
 - Function name (default: match_documents).
 - Embedding dimension (e.g., 1536 for text-embedding-3-large).noderecipe+1

Step 2: Build the ingestion workflow (once)

Create a new workflow: “RAG – Ingest Docs”.

1. **Trigger:** Manual (for testing) or Google Drive / Schedule.
2. **Source:**
 - For testing: use a **Form Trigger** or **Manual** with a file upload.
 - For automation: **Google Drive** node watching a folder.triggerworkflow+1
3. **Extract from File:** Convert PDFs/DOCX to text.
4. **Supabase Vector Store** node:
 - Operation: **Insert Documents**.
 - Add sub-nodes:
 - **Embeddings OpenAI** (choose your embedding model, e.g., text-embedding-3-large).
 - **Default Data Loader**.
 - **Text Splitter** (Recursive Character, chunk size ~1000 chars, overlap ~200).noderecipe+1
 - Configure:

- Table name, query function name.
 - Metadata: source (filename), doc_id, maybe category (FAQ, policy, etc.).noderecipe+1
5. Run once with a single document.
 6. Verify in Supabase: check the documents table has rows with non-empty content and embedding.noderecipe+1

Step 3: Build the chat/query workflow

Create a new workflow: “RAG – Support Chat”.

1. **Chat Trigger** (or Webhook / Telegram / WhatsApp trigger):
 - For testing: use n8n’s built-in **Chat Trigger**.
 - For multi-channel: later duplicate this workflow with different triggers but same core.triggerworkflow+1
2. **AI Agent** node:
 - System message (important):
“You are a support assistant. Answer only using the provided knowledge base context. If the context doesn’t contain the answer, say you don’t know.”noderecipe+1
3. Add sub-nodes to AI Agent:
 - **OpenAI Chat Model**:
 - Model: gpt-4o-mini (or your chosen model).
 - **Window Buffer Memory** (optional but recommended):
 - Session ID: from the Chat Trigger’s sessionId.
 - Context window length: e.g., 10–20 messages.triggerworkflow+1
 - **Supabase Vector Store** (as a tool):
 - Operation: **Retrieve Documents (As Tool for AI Agent)**.
 - Tool name: knowledge_base_search.
 - Tool description:
“Search our support knowledge base (policies, FAQs, product documentation) before answering any product, pricing, or policy question.”triggerworkflow+1
 - Top-k: 4–6.
 - Same embedding model as ingestion.

- Same Supabase table and function name.noderecipe+1

4. Connect:

- Chat Trigger → AI Agent (main).
- OpenAI Chat Model → AI Agent (ai_languageModel).
- Memory → AI Agent (ai_memory).
- Supabase Vector Store → AI Agent (ai_tool).triggerworkflow+1

5. Test:

- Ask a question that should be answerable from your docs.
- Check execution log: confirm the vector store tool was called and returned chunks.
- If the agent ignores the tool, improve the tool description.noderecipe+1

Step 4: Extend to multi-channel

To make this “multi-channel”:

- Duplicate the chat workflow.
- Replace **Chat Trigger** with:
 - **Telegram Trigger + Telegram Send Message.**
 - **WhatsApp Business Cloud** trigger + response node.
 - **Gmail Trigger + Gmail Send** for email support.
- Keep the **AI Agent + Supabase + OpenAI + Memory** core identical.
- Optionally add routing logic (e.g., classify intent, escalate to human) before/after the agent.aitoolly+2

Key design insights (why it’s built this way)

- **Two workflows (ingest vs chat):** Lets you update docs without touching the live bot; keeps costs and complexity separated.triggerworkflow+1
- **Agent + tool description:** The agent only searches your docs if it understands *when* to. A clear tool description is what makes RAG actually get used.triggerworkflow+2
- **Persistent vector store (Supabase):** In-memory is fine for demos, but real support needs durability across restarts and scale.triggerworkflow+2
- **Memory:** Makes conversations feel human; without it, every question is isolated.triggerworkflow+1

- **Chunking & embeddings consistency:** Same embedding model at ingest and query time is non-negotiable; mismatched models return nonsense with no error.